LAB–1: PYTHON PROGRAMMING BASICS

1.1 Aim

To study and implement the fundamentals of Python programming, including data structures, abstract data structures, and object-oriented programming concepts, as a foundation for Artificial Intelligence applications.

1.2 Theory Overview

List

Lists are ordered and mutable collections used to store dynamic sequences of data such as state lists and datasets.

```
# List example
states = ["Start", "A", "B"]
states.append("Goal")
print(states)
```

Tuple

Tuples are ordered but immutable collections used for fixed data such as coordinates or configurations.

```
# Tuple example
position = (3, 5)
print(position[0], position[1])
```

Set

Sets store unordered collections of unique elements and are useful for tracking visited states.

```
# Set example
visited = {"A", "B", "C"}
visited.add("D")
print(visited)
```

Dictionary

Dictionaries store key–value pairs and are widely used to represent mappings and heuristic tables.

```
# Dictionary example
heuristic = {"A": 5, "B": 3, "Goal": 0}
print(heuristic["B"])
```

Stack

A stack follows the LIFO principle and is used in recursion and depth-first search.

Python code for stack implementation.

1. The Stack class initializes an empty list self.items to store the stack items.
2. The push method adds an item to the end of the list self.items, which represents the top of the stack.
3. The pop method removes and returns the last item from the list self.items. If the list is empty, it returns None.
4. The peek method returns the last item from the list self.items without removing it. If the list is empty, it returns None.
5. The is_empty method returns True if the list self.items is empty, and False otherwise.

Example Code:

```
class Stack:

   def __init__(self):
      self.items = []   # Initialize empty stack
   def push(self, item):
      self.items.append(item)  # Add item to stack
   def pop(self):
      return self.items.pop() if self.items else None  # Remove top item
   def peek(self):
      return self.items[-1] if self.items else None    # View top item
  def is_empty(self):
      return not self.items
```

Queue

A queue follows the FIFO principle and is essential for breadth-first search algorithms.

Implementation of a queue in Python using a list

```
class Queue:
   def __init__(self):
      self.queue = []
   def enqueue(self, item):
      self.queue.append(item)
   def dequeue(self):
      if not self.is_empty():
         return self.queue.pop(0)
   def is_empty(self):
      return len(self.queue) == 0
  def size(self):  return len(self.queue)
```

Class and Object

Classes define blueprints for objects, which represent entities such as AI agents.

```
# Class and Object example
class Agent:
    def __init__(self, name):
        self.name = name

    def act(self):
        print(self.name, "is acting")

a = Agent("AI-Agent")
a.act()
```

Inheritance

Inheritance allows a class to acquire properties of another class, supporting hierarchical modeling.

```
# Inheritance example
class Agent:
    def act(self):
        print("Agent acting")

class LearningAgent(Agent):
    def learn(self):
        print("Learning")

la = LearningAgent()
la.act()
la.learn()
```

Encapsulation

Encapsulation restricts direct access to internal variables, improving data security.

```
# Encapsulation example
class Agent:
    def __init__(self, score):
        self.__score = score

    def get_score(self):
        return self.__score
```

Polymorphism

Polymorphism allows methods with the same name to perform different behaviors.

```
# Polymorphism example
class BFS:
    def search(self):
        print("BFS")

class DFS:
    def search(self):
        print("DFS")

for algo in (BFS(), DFS()):
    algo.search()
```

Abstraction

Abstraction hides implementation details and exposes only essential behavior.

```
# Abstraction example
from abc import ABC, abstractmethod

class SearchAlgorithm(ABC):
    @abstractmethod
    def search(self):
        pass

class AStar(SearchAlgorithm):
    def search(self):
        print("A* Search")

a = AStar()
a.search()
```

1.3 Lab Exercise:

Q1. Implementation of a queue in Python using two stacks.

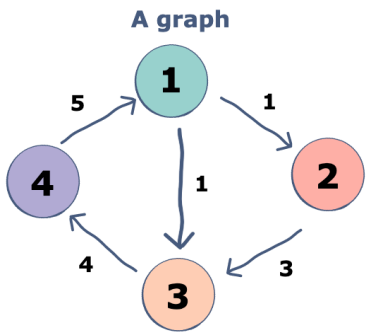
Description: A queue can be implemented using two stacks in Python by following the below steps:

1. Use two stacks, stack1 and stack2, to implement the enqueue and dequeue operations.
2. In the enqueue operation, push the new element onto stack1.

3. In the dequeue operation, if stack2 is empty, transfer all elements from stack1 to stack2. The element at the top of stack2 is the first element that was pushed onto stack1 and thus represents the front of the queue. Pop this element from stack2 to return it as the dequeued element.

Q2. Implement the following graph using python. Print the adjacency list and adjacency matrix.

[A graph is a data structure that consists of vertices that are connected via edges.]



Adjacency list	Adjacency matrix
{"1" : [[2, 1], [3,1]], "2" : [[3, 3]], "3" : [[4, 4]], "4" : [[1, 5]]}	[[0, 1, 1, 0], [0, 0, 3, 0], [0, 0 ,0 , 4], [5, 0, 0 ,0]]

Q3. Create two list X and Y with some set of numerical values. Compute Euclidean distance for corresponding values in X and Y. Store the distance values in a separate list and sort them using Bubble sort algorithm.

Formula

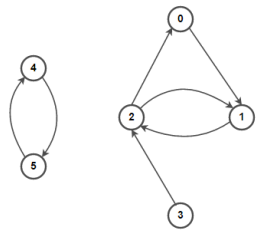
$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

p, q = two points in Euclidean n-space

q_i, p_i = Euclidean vectors, starting from the origin of the space (initial point)

n = n-space

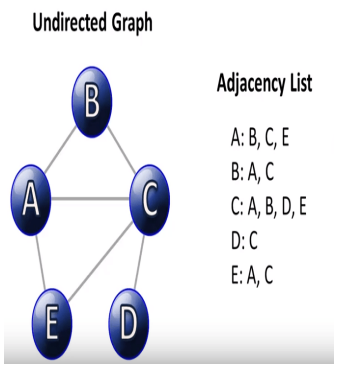
Q4. Implement the following directed unweighted graph using class, methods, and data structures of Python.



Expected output:

(0 —> 1)
(1 —> 2)
(2 —> 0) (2 —> 1)
(3 —> 2)
(4 —> 5)
(5 —> 4)

Q5. Implement the following undirected weighted graph using class, methods, and data structures of Python. Print the adjacency list and adjacency matrix.



Expected Output:

Adjacency List:

["A:['B',
'C', 'E']",
"C:['A',
'B', 'D',
'E']",

"B:['A',
'C', 'D']",
"E:['A',
'C']",
"D:['B',
'C']"]

Adjacency Matrix

```
[[ 0.  1.  1.  0.  1.]
 [ 1.  0.  1.  1.  0.]
 [ 1.  1.  0.  1.  1.]
 [ 0.  1.  1.  0.  0.]
 [ 1.  0.  1.  0.  0.]]
```